



Aprendizado por Reforço

Aula 7: API Open AI Gym e Gymnasium

Prof. Charles Madeira

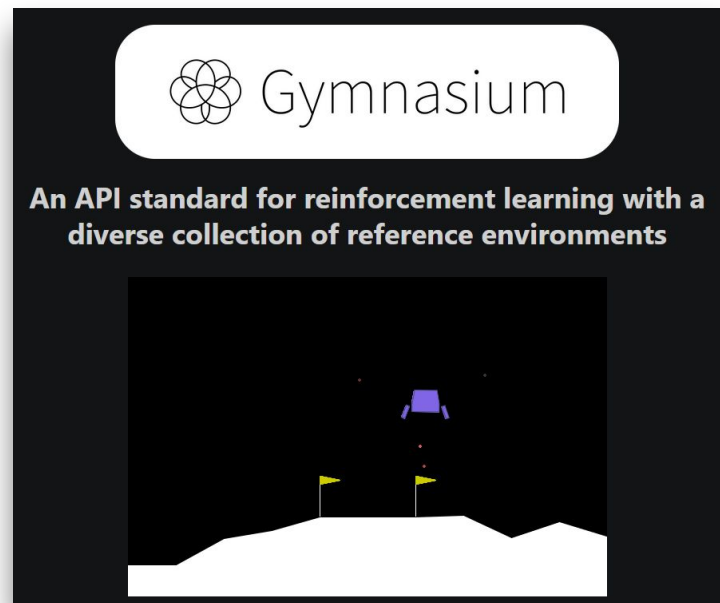
Objetivo da aula

- Introduzir a biblioteca Gymnasium, que é essencial para a iniciar a experimentação prática do Aprendizado por Reforço
- Detalhar os principais elementos da biblioteca
- Apresentar exemplos de código com um ambiente de controle clássico



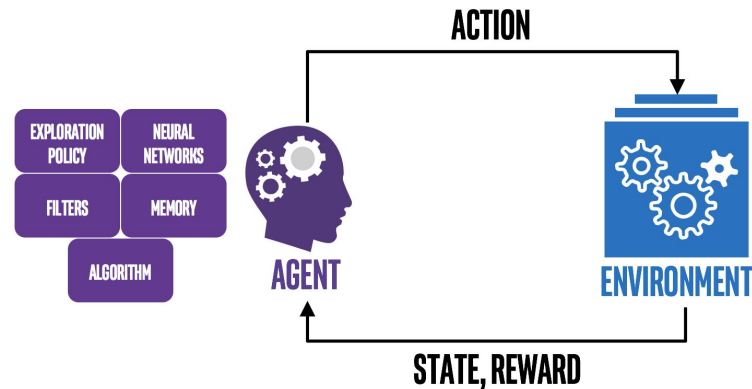
Introdução ao Gymnasium

- [Gymnasium](#) é uma biblioteca que fornece uma API uniforme para agentes de Aprendizado por Reforço (AR) e uma vasta coleção de ambientes
- A principal vantagem de ter uma API uniforme é a redução da necessidade de reescrever código padrão, permitindo a implementação de agentes de forma geral sem se preocupar com os detalhes do ambiente
- [Gymnasium](#) é um “fork” da biblioteca [OpenAI Gym](#), que foi desenvolvida pela OpenAI em 2017 e se tornou um padrão para AR, implementando a mesma API para ser um substituto direto
 - *OpenAI Gym* não é mais mantida atualmente



Conceitos Fundamentais: Agente e Ambiente

- Aprendizado por Reforço se baseia em dois componentes principais:
 - **Agente**: entidade que assume um papel ativo para implementar alguma política
 - decide qual ação deve ser executada a cada passo de tempo, dada a sua observação do estado do ambiente
 - tem como resultado uma transição de estado e uma recompensa
 - **Ambiente**: tudo o que é externo ao agente
 - tem a responsabilidade de fornecer observações e atribuir recompensas
 - muda de estado com base nas ações do agente



Anatomia de um Ambiente (*Environment*) Simples

- **Estado Interno:** Mantém um *steps_left* para limitar o número de interações
- **get_observation():** Retorna a observação atual, que é geralmente implementada como alguma função do estado interno do ambiente (ex.: vetor de zeros, pois o ambiente não tem estado interno complexo)
- **get_actions():** Permite ao agente consultar o conjunto de ações que pode executar (ex.: [0, 1])
- **is_done():** Sinaliza o fim do episódio (ex.: *self.steps_left* nulo)
- **action(action: int):** Peça central que realiza duas tarefas
 - Lida com a ação do agente e retorna um reforço para esta ação (ex.: *random.random()*)
 - Atualiza a contagem de passos e lança uma exceção se o episódio acabar

```
class Environment:
    def __init__(self):
        self.steps_left = 10

    def get_observation(self) -> List[float]:
        return [0.0, 0.0, 0.0]

    def get_actions(self) -> List[int]:
        return [0, 1]

    def is_done(self) -> bool:
        return self.steps_left == 0

    def action(self, action: int) -> float:
        if self.is_done():
            raise Exception("Game is over")
        self.steps_left -= 1
        return random.random()
```

Anatomia de um Agente (*Agent*) Simples

- **Estado Interno**: Mantém um *total_reward* acumulado
- **step(env: Environment)**: Realiza um passo no ambiente, englobando as seguintes ações:
 - **Observar o ambiente**
 - `env.get_observation()`
 - **Tomar uma decisão** sobre a ação a ser executada
 - Ex.: `random.choice(actions)`
 - **Submeter a ação** ao ambiente
 - `env.action(...)`
 - **Obter uma recompensa** pelo passo atual e adicioná-la a *total_reward*

```
class Agent:
    def __init__(self):
        self.total_reward = 0.0

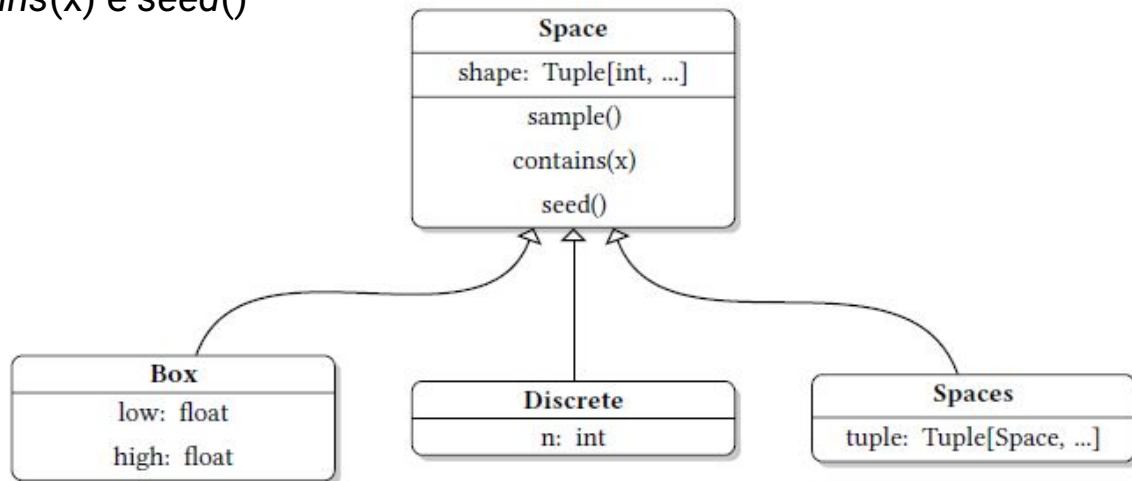
    def step(self, env: Environment):
        current_observation = env.get_observation()
        actions = env.get_actions()
        reward = env.action(random.choice(actions))
        self.total_reward += reward
```

Componentes Essenciais do Ambiente (Env)

- **Espaço de Ações**: suporta ações discretas, contínuas ou uma combinação
 - **Ações Discretas**: conjunto fixo e mutuamente exclusivo de ações, representadas pela classe **Discrete**($n=X$)
 - Ex.: esquerda, direita, cima, baixo
 - **Ações Contínuas**: ações com valores associados e limites, representadas pela classe **Box**(*low, high, shape, dtype*)
 - Ex.: ângulo do volante, pressão do acelerador
 - **Múltiplas Ações**: classe **Tuple** que permite aninhar vários espaços de ações para ações simultâneas

Componentes Essenciais do Ambiente (Env)

- **Espaço de Estados**: observações que o ambiente fornece ao agente em cada passo de tempo
 - Pode ser números simples, tensores multidimensionais (imagens), ou mesmo valores discretos (ex.: uma lâmpada “ligada/desligada”)
 - Representadas de forma semelhante às ações pelas classes **Discrete**, **Box** e **Tuple**
 - A classe base **Space** inclui propriedades, tais como *shape*, e métodos, tais como *sample()*, *contains(x)* e *seed()*



Componentes Essenciais do Ambiente (Env)

- `reset()`: método que leva o ambiente ao seu estado inicial e retorna uma tupla de dois elementos, devendo ser chamado após a criação do ambiente e após o final de cada episódio
 - `observation`: vetor ou matriz NumPy com dados da primeira observação
 - `info`: dicionário com informações extras específicas do ambiente (geralmente ignorado em métodos gerais de AR)
- `step()`: método para execução de uma ação, que retorna uma tupla de cinco elementos
 - `observation`: vetor ou matriz NumPy com dados da observação atual
 - `reward`: float do reforço recebido
 - `terminated`: booleano que indica `True` quando o episódio termina naturalmente
 - `truncated`: booleano que indica `True` quando o episódio atinge um limite de passos
 - `info`: dicionário com informações extras específicas do ambiente

```
# Reset the environment
observation, info = env.reset()

# Perform steps and observe the reward
while True:
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    total_steps += 1
    if terminated or truncated:
        break
```

Criação de Ambientes

- Ambientes são criados usando a função `gymnasium.make(name)`, em que **name** é uma string no formato ***EnvironmentName-vN*** (nome do ambiente e versão)

```
import gymnasium as gym

# Create a Gym environment
env = gym.make("CartPole-v1")
total_reward = 0.0
total_steps = 0
```

- O Gymnasium v1.3.0 contém mais 200 ambientes únicos, chegando a mais de 1000 ambientes se considerarmos as variações de versões
 - Exemplo do ambiente do jogo *Breakout* do Atari com suas diversas variações
 - *Breakout-v0, Breakout-v4, BreakoutDeterministic-v4, Breakout-ram-v0,...*

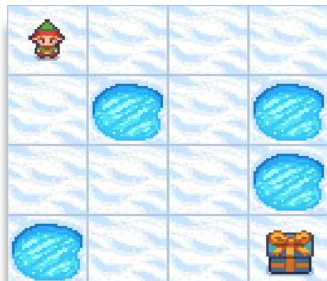
Categorias de Ambientes

- **Classic Control**: problemas de controle clássicos
- **Box2D**: jogos baseados em simulação de física via box2D
- **Toy Text**: ambientes com espaços de estados e ações discretizados
- **MuJoCo**: mecanismos físicos com controle multi-articular
- **Atari 2600**: jogos do Atari 2600
- **Third-party**: diversos ambientes compatíveis com o Gymnasium

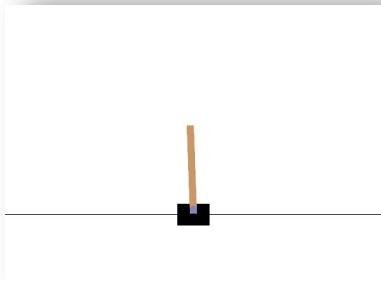
Atari 2600



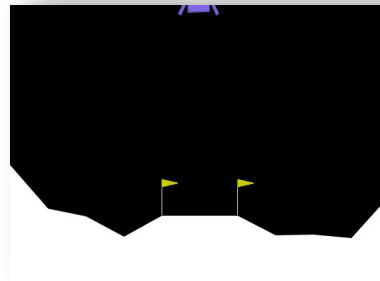
Toy Text



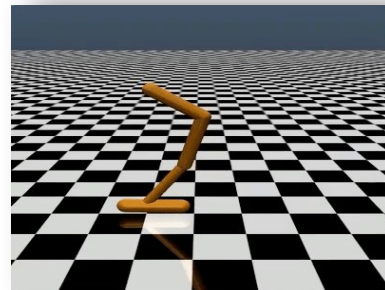
Classic Control



Box 2D

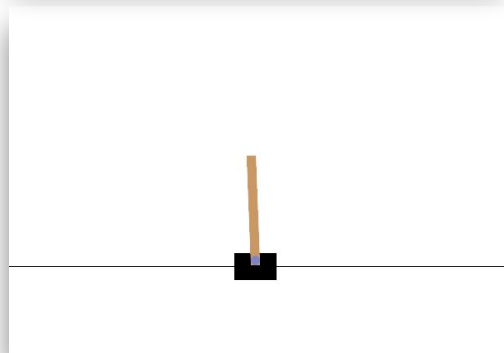
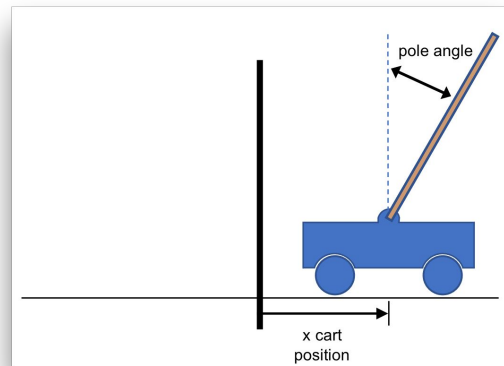


MuJoCo



Exemplo do Ambiente CartPole-v1

- **CartPole-v1** é um ambiente de **controle clássico** (“Hello World”) do Aprendizado por Reforço, no qual o objetivo do agente é manter um pêndulo equilibrado em uma plataforma móvel a fim de maximizar o seu reforço
 - Um agente aleatório dura em média 12-15 passos, enquanto a recompensa limite para “resolver” o ambiente é 195 passos
- A **observação** consiste em quatro valores de ponto flutuante
 - (1) Posição do carrinho, (2) velocidade do carrinho, (3) ângulo do pêndulo, (4) velocidade angular do pêndulo
 - **observation_space** é do tipo **Box(4,)**
- O **conjunto de ações** consiste em duas ações discretas
 - **0** para mover para a esquerda, **1** para mover para a direita
 - **action_space** é do tipo **Discrete(2)**
- O **reforço** é de 1.0 a cada passo, enquanto o poste não cai



Exemplo de Agente CartPole Aleatório

- Código para um agente aleatório no CartPole-v1 que demonstra a simplicidade da interação com o Gymnasium

```
import gymnasium as gym

# Create a Gym environment
env = gym.make("CartPole-v1")
total_reward = 0.0
total_steps = 0

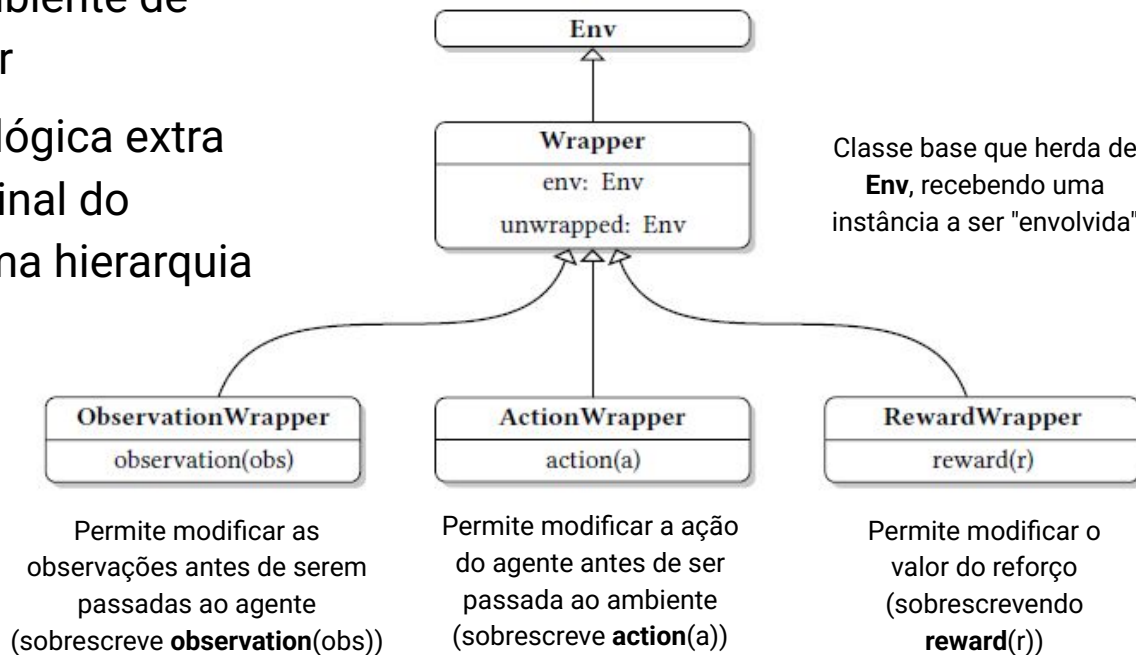
# Reset the environment
observation, info = env.reset()

# Perform steps and observe the reward
while True:
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    total_steps += 1
    if terminated or truncated:
        break

print("Episode done in %d steps, total reward %.2f" % (total_steps, total_reward))
```

Funcionalidades Extras: Wrappers

- **Wrappers** são ferramentas poderosas para estender ou modificar a funcionalidade de um ambiente de forma genérica e modular
- Eles permitem adicionar lógica extra sem alterar o código original do ambiente, por meio de uma hierarquia



Exemplo de ActionWrapper para Exploração

- Um [ActionWrapper](#) pode ser usado para introduzir exploração, substituindo a ação do agente por uma ação aleatória com uma probabilidade epsilon, o que consiste em um dos métodos mais práticos e poderosos para resolver o problema de exploração/aproveitamento

```
class RandomActionWrapper(gym.ActionWrapper):
    def __init__(self, env: gym.Env, epsilon: float = 0.1):
        super(RandomActionWrapper, self).__init__(env)
        self.epsilon = epsilon

    def action(self, action: gym.core.WrapperActType) -> gym.core.WrapperActType:
        if random.random() < self.epsilon:
            action = self.env.action_space.sample()
            print(f"Random action {action}")
            return action
        return action
```

Exemplo de Agente CartPole com ActionWrapper

- Código para um agente CartPole-v1 com *ActionWrapper* que demonstra a simplicidade da interação com o Gymnasium

```
# Create a Gym environment with RandomActionWrapper
env = RandomActionWrapper(gym.make("CartPole-v1"))

# Reset the wrapped environment
observation, info = env.reset()
total_reward = 0.0

# Perform steps and observe the reward
while True:
    observation, reward, terminated, truncated, info = env.step(0)
    total_reward += reward
    if terminated or truncated:
        break

print(f"Reward got: {total_reward:.2f}")
```


Renderização do Ambiente

- Gymnasium oferece *wrappers* para visualizar o ambiente, se mostrando útil para depuração e visualização do comportamento do agente, especialmente em máquinas sem GUI
 - *HumanRendering*: abre uma janela gráfica interativa para exibir o ambiente, requerendo que o ambiente seja inicializado com **render_mode="rgb_array"**
 - *RecordVideo*: captura os pixels do ambiente e produz um arquivo de vídeo da ação do agente, requerendo que o ambiente seja inicializado com **render_mode="rgb_array"** e um argumento **video_folder**

```
# Create a Gym environment that supports rgb_array rendering
env = gym.make("CartPole-v1", render_mode="rgb_array")

# Wrap the environment with RecordVideo to save videos
wrapped_env = gym.wrappers.RecordVideo(env, video_folder="./videos")

...

# Close the environment to finalize video saving
wrapped_env.close()
```

Requisitos de Hardware e Software

- **Linguagem:** [Python](#) na versão 3.1x
- **Bibliotecas Externas de código aberto:**
 - [Gymnasium](#) (*Farama Foundation*): fork da biblioteca *OpenAI Gym* que dispõe de um framework contendo vários ambientes que podem ser utilizados de forma unificada
 - [NumPy](#): Para computação científica e operações com matrizes
 - [OpenCV Python bindings](#): Para visão computacional e processamento de imagens
 - [PyTorch](#): biblioteca flexível de Aprendizado Profundo
 - [PyTorch Ignite](#): Ferramentas de alto nível sobre o PyTorch para reutilização de código padrão
- **Hardware:** Uso recomendado de GPUs para os experimentos devido ao aprendizado profundo ser computacionalmente custoso
 - GPUs chegam a ser de 10 a 100 vezes mais rápidas que as melhores CPUs
 - [Google Colab](#) é uma opção interessante para diversos experimentos

Exemplo do CartPole disponível no Colab

- Experimentar o Gymnasium com o CartPole a partir da implementação no [Colab](#) disponível no Drive da disciplina



- Também experimentar outros ambientes integrados ao Gymnasium